Smart Factory Equipment Anomaly Alert Agent

Pegatron ML Engineer Assignment — Assignment 3
Vaibhav Kumar Sunkaria · Built end-to-end with free AI tools · https://github.com/vaibhavsunkaria97/smart-factory-agent

1 Task Description & Objective

An AI agent that monitors smart-factory equipment sensor data, detects anomalies automatically, and delivers **actionable real-time alerts**. The pipeline generates a dummy sensor dataset (temperature, pressure, vibration), cleans it, scores every reading with two independent detectors, explains each anomaly through an LLM, and emits a prioritised alert table plus a machine-readable JSON log for downstream MES integration.

Design constraint taken from the brief. Because the assignment rewards a low usage barrier, the system runs with **one command and no API key**: `python run_all.py` regenerates the data, alerts, metrics, figures and test results. LLM reasoning is used when available and degrades gracefully to a deterministic expert reasoner when it is not.

2 Development Workflow (AI-driven)

The project was built with an **agentic coding loop**: Aider driving a free-tier frontier model, reading the repository, writing files and committing to git. Manual effort was limited to specifying constraints, verifying every output by execution, and taking over where the agent stalled. Full prompts and the defect log are in `docs/AI_LOG.md`.

3 Method & Model Details

3.1 Data generation and preprocessing

300 rows at one-minute intervals, ~11% abnormal, with 1–3 channels corrupted per abnormal row. Missing values and duplicate timestamps are injected deliberately so cleaning is not a no-op. A generator-side assertion guarantees every row labelled abnormal breaches at least one threshold. Preprocessing de-duplicates, imputes with time-aware interpolation, and produces a **separate z-scored copy** for the ML detector — physical units are kept apart from scaled values so alerts stay human-readable.

3.2 Detection ensemble

A **rule detector** applies the brief's physical thresholds (temp >52/<43 °C, pressure >1.08/<0.97 bar, vibration >0.07 g) — fast, explainable, no training. An **IsolationForest** (200 trees) learns the normal multivariate envelope unsupervised and catches patterns no single threshold expresses. Scores are fused as an OR-of-evidence (`score = max(rule, ML)`), favouring recall because a missed fault costs more than a false alarm. Severity follows from breach count and score.

3.3 Reasoning layer

Each anomaly is turned into a diagnosis and recommended action by the best available free backend — local **Ollama**, free-tier **Groq**, or a deterministic knowledge base keyed on (signal, direction). The LLM never alters a detection; it only phrases the explanation, so correctness remains deterministic and unit-tested.

4 System Architecture

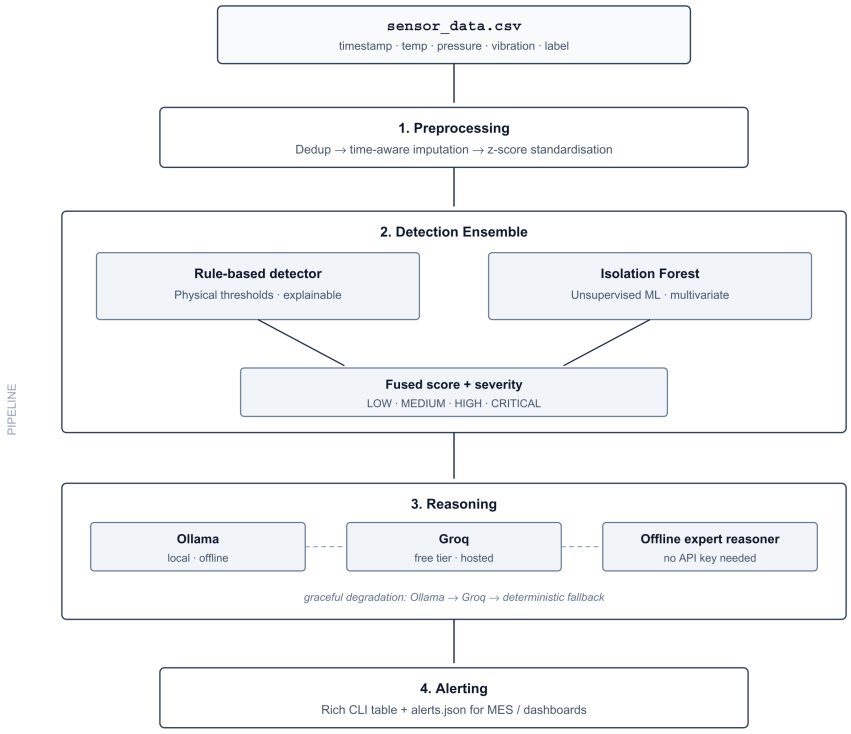

Fig. 1 — Smart Factory Agent processing pipeline

Figure 1 — Four-stage pipeline with graceful degradation at the reasoning layer.

5 Evaluation

Four configurations were compared against ground-truth labels (300 readings, 33 true anomalies). F2 is reported alongside F1 because β=2 weights recall four times more than precision, which is the right emphasis for equipment safety. Accuracy is deliberately omitted: with ~11% positives, predicting "all normal" would already score ~89%.

Detector	Precision	Recall	F1	F2	Flagged
Rule-based only	0.943	1.000	0.971	0.988	35
IsolationForest only	0.917	1.000	0.957	0.982	36
Gaussian Mixture (K=2)	0.917	1.000	0.957	0.982	36
Rule + IF ensemble (shipped)	**0.917**	**1.000**	**0.957**	**0.982**	**36**

Table 1 — Detector comparison. All configurations achieve perfect recall.

An honest finding: preprocessing manufactured two anomalies. Precision is below 1.0 for every detector, and investigation showed why. Two rows with a missing vibration reading sat between high-vibration neighbours; time-interpolation filled them with 0.080 and 0.088, both above the 0.07 limit. Those rows are labelled normal but breach after cleaning, so they count as false positives. The detectors behaved correctly — the imputed value never occurred. **Cleaning is not a neutral step.** In production the fix is causal forward-fill (which the streaming path requires anyway) plus flagging imputed values in the alert so an operator knows the reading was inferred rather than measured.

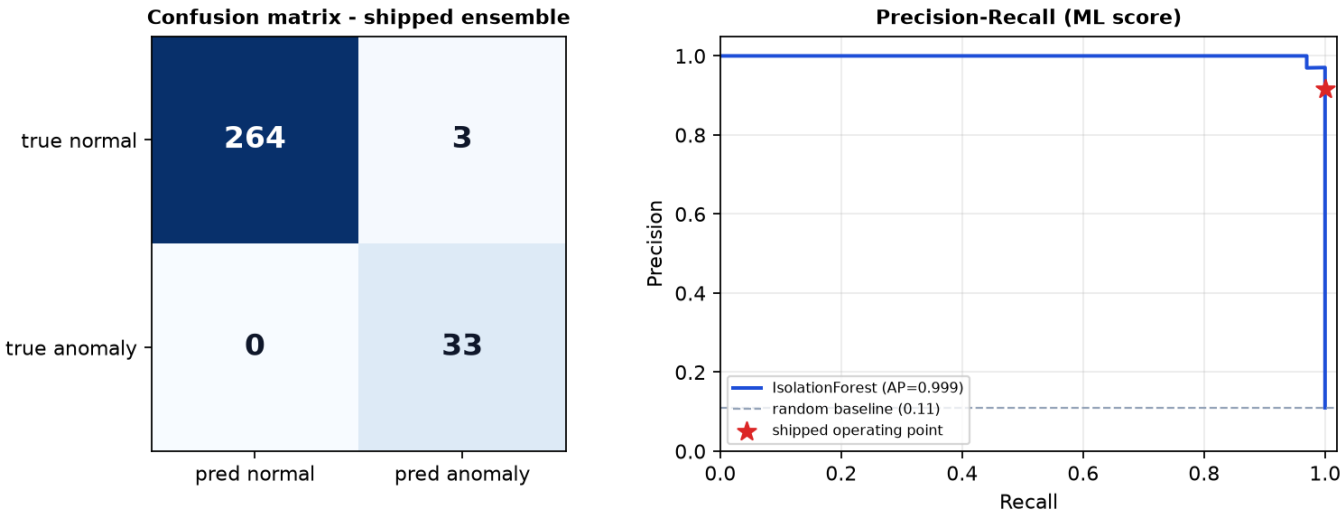

Figure 2 — Confusion matrix and precision–recall curve with the shipped operating point.

6 Demo

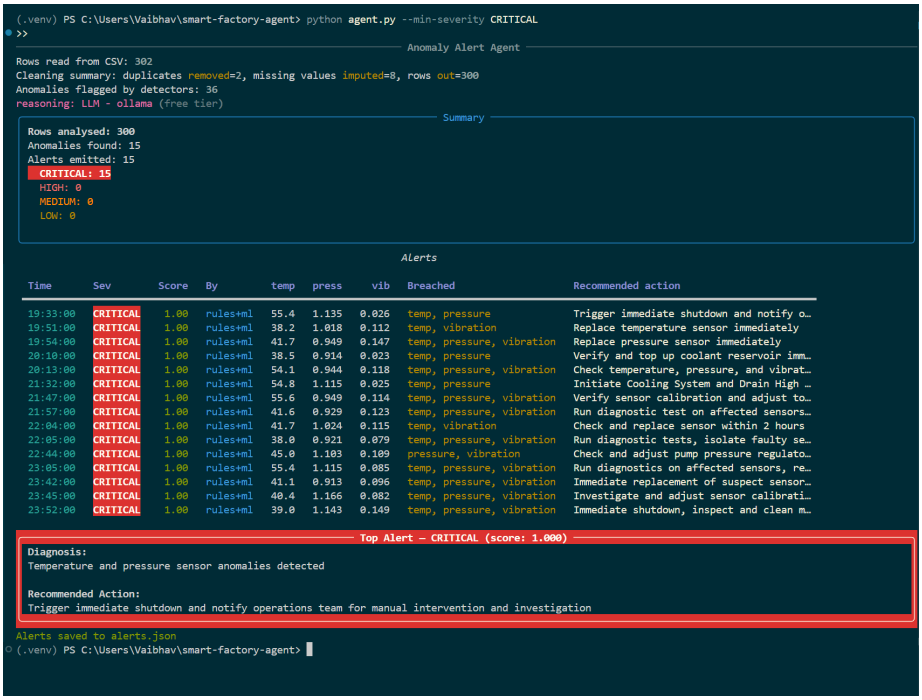

Figure 3 — CRITICAL alerts with live Ollama reasoning: cleaning summary, prioritised table, and the top-priority alert card.

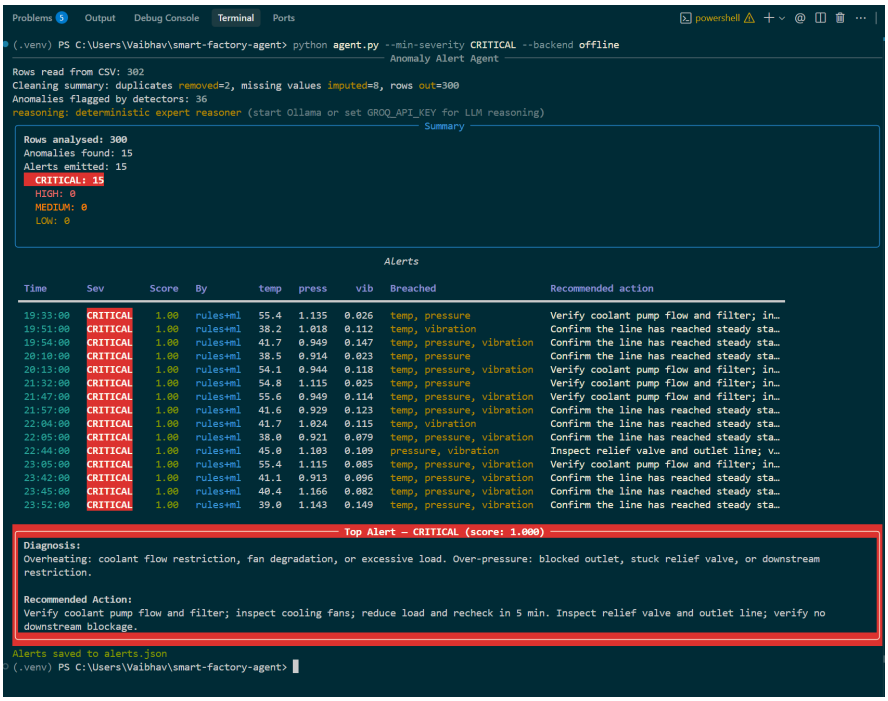

Figure 4 — Offline fallback execution with deterministic expert reasoner output when no LLM backend is active.

7 Use of AI Tools

Only free tiers were used. AI appears in two distinct roles: tools that **built** the system, and an LLM embedded **inside** the delivered product as its reasoning layer.

Free AI tool	Type	Contribution
Aider + free-tier frontier model	Build-time	Agentic loop: read repo, planned changes, wrote modules and tests, committed to git.
Claude (free tier)	Build-time	Requirement analysis, architecture decisions, debugging, report generation.

Ollama / llama3.2 (local)	Runtime	LLM inside the product writing each alert's diagnosis — free, offline, air-gap friendly.
Groq (free tier)	Runtime	Hosted alternative reasoning backend.

7.1 AI as reviewer, not only generator

Every AI output was verified by execution, and verification caught defects that reading the code did not: mislabelled rows where 8 of 41 'abnormal' readings breached no threshold; a syntax error from a duplicated loop header; a runtime TypeError from interpolating a string column; a scale-dependent bug where 100-row datasets received zero missing values; and — most instructive — an LLM backend that reported success while every call silently failed, because it posted to the wrong endpoint and a bare `except: pass` hid it. The status line said "ollama" while zero LLM calls had succeeded. Report observed state, never intended state.

Measured limits of the local model. On multi-breach rows llama3.2 frequently names only one fault, occasionally inverts the direction (recommending "increase temperature" for a low-temperature fault), and has hallucinated non-existent hardware ("Replace sensor 3"). The deterministic knowledge base never does. This is the empirical basis for keeping detection deterministic and confining the LLM to phrasing.

8 Limitations & Next Steps

The system detects **point anomalies** only; contextual and collective (subsequence) faults would need windowed or sequence models — deliberately not attempted here because the synthetic data contains independent point anomalies with no temporal structure for a sequence model to learn. Batch imputation is non-causal and unsuitable for streaming. Severity cutoffs are a heuristic rather than a derivation. Next steps: Kafka ingestion in place of CSV, per-equipment learned thresholds, drift detection with retraining, an operator confirm/dismiss feedback loop to build real labels, retrieval-grounded maintenance actions, and MES integration so a CRITICAL alert opens a work order.